This document covers the design and development of a simple pizza ordering system for Pennine Pizza Works. The final system is designed to make order calculations more consistent and reduce errors when processing pizza orders.

PENNINE PIZZA ORDERING SYSTEM

COM4401: Foundations of Computer Science and Academic Skills
Software Engineering Fundamentals

Chiamaka Eluwa, Enrique Manzano Ayala, Toby Mathew, Fatimah Bibi.

Table of Contents

1. Introduction 3
1.1. Business Context 3
1.2. Project Aim 3
1.3. Development Approach 3
2. Analysis 4
2.1. Business Rules 4
2.2. Requirements 4
2.3. Input – Process – Output (IPO) Table 5
3. Design 7
3.1. Structure Chart (Decomposition) 7
3.2. System Functions & Subroutines 7
3.2.1. Overview 7
3.2.2. Data Plan (Arrays & Variables) 8
3.2.3. Subroutine List 9
3.2.4. Design Decisions 11
3.3. Main Flowchart Design 12
3.4. Pseudocode (Generated with Flowgorithm) 13
4. Implementation 17
4.1. Flowgorithm Version History 17
4.2. Program Flowcharts 18
4.2.1. Main 18
4.2.2. GetCustomerInfo 19
4.2.3. GetPizzaCount 19
4.2.4. GetPizzaChoices 20
4.2.5. GetDeliveryOption 20
4.2.6. GetSizePrice 21
4.2.7. GetToppingCharge 21
4.2.8. CalculatePizzaCosts 22

4.2.9. CalculateSubtotal 22
4.2.10. CalculateDiscount 23
4.2.11. GetDeliveryCharge 23
4.2.12. CalculateFinalTotal 24
4.2.13. DisplayCustomerHeader 24
4.2.14. DisplayItemisedPizzas 25
4.2.15. DisplayTotals 25
4.2.16. DisplayFinalTotal 26
5. Testing 26
5.1. Test Table 27
5.2. Issues Found and Fixes 31
5.3. Limitations and Future Improvements 32

1. Introduction

1.1. Business Context

Pennine Pizza Works is a small independent pizza takeaway that currently takes orders over the phone and at the counter. Staff write the customer's order down on paper and then use a calculator to calculate the total price. This can become difficult and lead to mistakes especially with multiple pizzas, extra toppings, discounts and delivery changes. This business therefore needs a simple ordering system that can handle these calculations accurately and make the ordering process easier for staff.

1.2. Project Aim

The aim of this project is to create a simple pizza ordering system using Flowgorithm. We developed the system by first analysing the requirements, designing the logic on Draw.io, building it in Flowgorithm, testing it with different types of data, and making improvements where problems are found.

1.3. Development Approach

We followed the **Waterfall** life cycle for this project. This means we finished one stage before moving on to the next: Analysis, Design, Implementation (build), Testing and then Deployment. We chose it because the brief is clear and does not change, and because every stage of the brief is asked for in this order.

Stage	What we did	Where in this report
Analysis	Read the brief, wrote the business rules, requirements (R1–R12) and the IPO table.	Section 2
Design	Structure chart, data plan, subroutine list, main flowchart and pseudocode.	Section 3
Implementation	Built the program in Flowgorithm as 16 functions (Main plus 15 subroutines).	Section 4
Testing	Tested with normal, boundary and abnormal data, and logged the issues we found.	Section 5
Deployment	Final version, GitHub README and instructions for running it.	GitHub repository

2. Analysis

2.1. Business Rules

The business rules define the fixed pricing and ordering conditions that the system must follow. These rules are reflected in the requirements table and will be used when designing the calculations and testing whether the program works as intended.

Item	Rule
Minimum number of pizzas	1 pizza
Maximum number of pizzas	6 pizzas
Small pizza	£3.25
Medium pizza	£5.50
Large pizza	£7.15
1 extra topping	£0.75
2 extra toppings	£1.35
3 extra toppings	£2.00
4 or more extra toppings	£2.50
Discount	10% if the order subtotal is over £20
Delivery charge	£2.50
Discount calculation	The discount is subtracted from the order subtotal before the delivery charge is added.

2.2. Requirements

The requirements describe what the pizza ordering system must do, in that it identifies what the system must allow the user to enter, what the program must calculate, and what it must produce, as defined in the 'Type' field. Each requirement has been given a unique ID so that it can be referred to during the design, development and testing stages.

ID	Requirement	Description	Type
R1	Customer details	The program must allow staff to enter the customer's name, address, and phone number.	Input
R2	Number of pizzas	The program must allow between 1 and 6 pizzas to be ordered.	Input
R3	Pizza size	The system must allow the size of each pizza to be entered as small, medium, or large.	Input

R4	Extra toppings	The system must allow the number of extra toppings for each pizza to be entered.	Input
R5	Delivery	The system must allow staff to enter whether delivery is required.	Input
R6	Pizza costs	The program must calculate the cost of each pizza.	Process
R7	Topping costs	The program must calculate the extra topping cost for each pizza.	Process
R8	Subtotal	The program must calculate the order subtotal before discount and delivery	Process
R9	Discount	The program must apply the 10% discount when the order subtotal is over £20.	Process
R10	Delivery charge	The program must apply the £2.50 delivery charge when delivery is requested.	Process
R11	Final total	The program must calculate and display the final total cost of the order.	Output
R12	Itemised bill	The program must produce a personalised and itemised bill of the order.	Output

2.3. Input – Process – Output (IPO) Table

The IPO table shows how information would move through the pizza ordering system. It would identify the information or data entered by the staff, the calculations and decisions made by the program, and the information produced at the end of the order.

DATA/ VARIABLE	INPUT	PROCESS	OUTPUT
Customer name	E.g., Amaka, Toby, Beth.	Store customer name for use in the order	N/A
Customer address	Enter customer address	Store customer address for use in the order	N/A
Customer phone number	Enter customer phone number	Store customer phone number for use in the order	N/A
Number of pizzas	E.g., 1, 2… 6.	Check that the number is between 1 and 6. If not, ask the user to enter it again. Store the valid number.	N/A
Pizza size	E.g., Small, Medium, or Large	Match the selected size to its price	N/A
Topping quantity	E.g., 0, 1, 2, 3 or 4+	Match the number of toppings to the correct charge	N/A

Cost of each pizza	N/A	Add the pizza price and topping charge. Repeat for each pizza until all pizzas are processed	Cost of each pizza
Subtotal	N/A	Add all the individual pizza costs together	Subtotal
Total after discount	N/A	Check whether the subtotal is over £20. If yes, calculate a 10% discount and subtract it from the subtotal. If no, set the discount to £0.00.	Total after discount
Delivery charge	Enter Yes or No	If Yes, set the delivery charge to £2.50. If No, set the delivery charge to £0.00.	Delivery charge
Final total	N/A	Add the delivery charge to the total after discount	Final total
Itemised bill	N/A	Combine the stored information and calculated values into an itemised bill	Display the completed itemised bill including customer details, the cost of each pizza, order subtotal, total after discount, delivery charge and final total (2 d.p.).

P.T.O

3. Design

3.1. Structure Chart (Decomposition)

Decomposition means breaking a large problem into smaller parts that are easier to design, build, and test. The structure chart in *Figure 1* below does this for the pizza ordering system. It starts with the whole system at the top and splits it into four main modules each of which is split again into individual tasks. The chart was created using draw.io

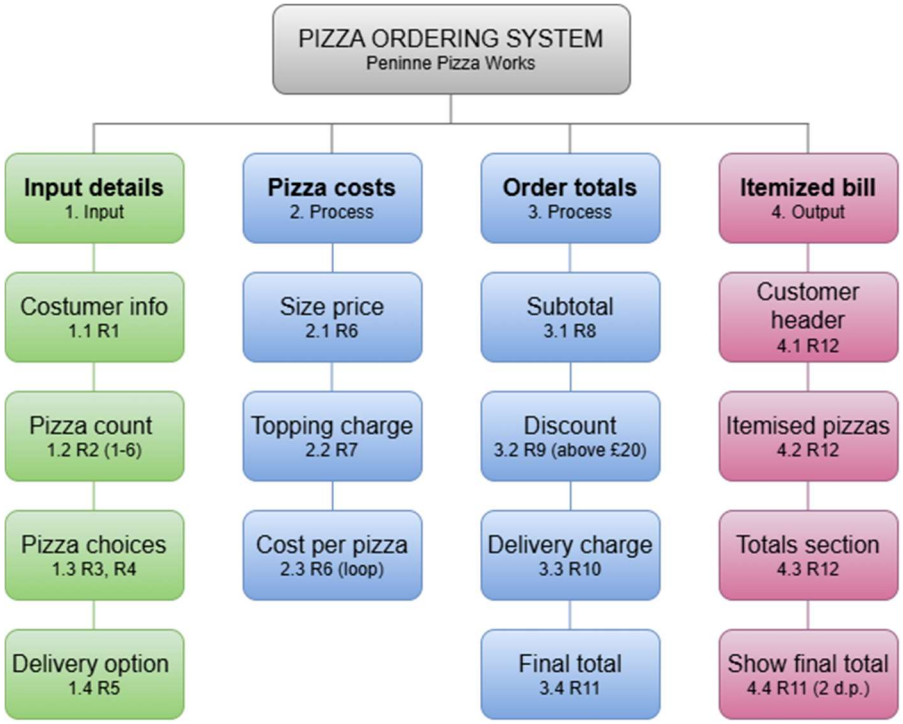


Figure 1: Structure chart for Pennine Pizza Works ordering system

3.2. System Functions & Subroutines

3.2.1. Overview

The system is split into subroutines. A subroutine is a small, named block of steps that does one job and can be used whenever it is needed, thus making the program easier to read, test, and fix. Each subroutine matches a box on the structure chart in 3.1., so every requirement (R1 to R12) can be traced to the subroutine that meets it.

Two types are used:

- **Functions** take in values, calculate something and return a single result. For example, GetSizePrice returns the price of a pizza size.
- **Procedures** carry out a task, like reading input or displaying output, without returning a value. Where they need to pass several values, they store them in arrays.

3.2.2. Data Plan (Arrays & Variables)

Arrays

Array	Type	Size	Declared In	Holds	Example
customerDetails[]	String	3	Main	[0] name, [1] address, [2] phone number. Filled by **GetCustomerInfo**	customerDetails[0] = "Amaka"
sizes[]	String	6	Main	The size chosen for each pizza. Filled by **GetPizzaChoices**	sizes[1] = "Large"
toppings[]	Integer	6	Main	The number of extra toppings on each pizza. Filled by **GetPizzaChoices**	toppings[1] = 2
sizePrices[]	Real	6	Main	The standard price of each pizza from **GetSizePrice**. Filled by **CalculatePizzaCosts**.	sizePrices[1] = 7.15
toppingCharges[]	Real	6	Main	The extra topping cost of each pizza from **GetToppingCharge**. Filled by **CalculatePizzaCosts**.	toppingCharges[1] = 1.35
pizzaCosts[]	Real	6	Main	The total cost of each pizza (size price + topping charge)	pizzaCosts[1] = 8.50

Variables declared in Main

Variable	Type	Passed to	Valid Values
numPizzas	Integer	**GetPizzaChoices, CalculatePizzaCosts, CalculateSubtotal, DisplayItemisedPizzas**	1 to 6
delivery	String	**GetDeliveryCharge**	"Yes" or "No"
subtotal	Real	**CalculateDiscount, CalculateFinalTotal, DisplayTotals**	Greater than 0
discount	Real	**CalculateFinalTotal, DisplayTotals**	0.00 or 10% of subtotal
deliveryCharge	Real	**CalculateFinalTotal, DisplayTotals**	0.00 or 2.50
finalTotal	Real	**DisplayFinalTotal**	Shown to 2 d.p.

Local variables (exist only inside their subroutine)

Variable	Type	Declared In	Purpose	Valid Values
count	Integer	**GetPizzaCount**	Holds the number of pizzas typed in	1 to 6 (repeats until valid)
counter	Integer	**GetPizzaChoices, CalculatePizzaCosts, CalculateSubtotal, DisplayItemisedPizzas**	Loop counter, one pass per pizza	0 to numPizzas – 1 (max 5)
deliveryChoice	String	**GetDeliveryOption**	Holds the Yes/No typed in	“Yes” or “No”
sizePrice	Real	**GetSizePrice**	Price of the size, which is returned	3.25, 5.50 or 7.15

toppingCharge	Real	**GetToppingCharge**	Charge for the toppings, which is returned	0.00, 0.75, 1.35, 2.00 or 2.50
subtotal	Real	**CalculateSubtotal**	Running total of the pizza costs. It starts at 0 and is returned	Greater than 0
discount	Real	**CalculateDiscount**	The discount, which is returned	0.00 or 10% of subtotal
deliveryCharge	Real	**GetDeliveryCharge**	The charge, which is returned	0.00 or 2.50
finalTotal	Real	**CalculateFinalTotal**	The final total, which is returned	(subtotal – discount) + deliveryCharge

The calculation subroutines each return their result to Main, where it is stored in the Main variable with the same name. **GetCustomerInfo**, **DisplayCustomerHeader**, **DisplayTotals** and **DisplayFinalTotal** have no local variables. size and toppingCount are parameters, not declared variables.

3.2.3. Subroutine List

Ref	Subroutine	Type	Parameters (In)	Returns (Out)	Purpose	Req
1. Input details						
1.1	**GetCustomerInfo**	Procedure	customerDetails[]	Fills customerDetails[]	Prompts for the name, address, and phone number and stores them in the array	R1
1.2	**GetPizzaCount**	Function	None	count (Integer)	Asks for the number of pizzas and repeats until the value is between 1 and 6	R2
1.3	**GetPizzaChoices**	Procedure	numPizzas, sizes[], toppings[]	Fills sizes[] and toppings[]	Loops once per pizza to enter the size (small, medium or large) and the number of extra toppings	R3, R4
1.4	**GetDeliveryOption**	Function	None	deliveryChoice (String)	Asks whether delivery is required ("Yes" or "No")	R5
2. Pizza costs						
2.1	**GetSizePrice**	Function	size	sizePrice (Real)	Matches the size to its price: Small £3.25, Medium £5.50, Large £7.15	R6

2.2	**GetToppingCharge**	Function	toppingCount	toppingCharge (Real)	Matches the topping count to its charge: 0 = £0.00, 1 = £0.75, 2 = £1.35, 3 = £2.00, 4 or more = £2.50	R7
2.3	**CalculatePizzaCosts**	Procedure	numPizzas, sizes[], toppings[], sizePrices[], toppingCharges[], pizzaCosts[]	Fills sizePrices[], toppingCharges[], and pizzaCosts[]	Loops through each pizza, calls 2.1 and 2.2, stores both values, and adds them to give the cost of each pizza	R6, R7
3. Order totals						
3.1	**CalculateSubtotal**	Function	numPizzas, pizzaCosts[]	subtotal (Real)	Adds all the individual pizza costs together	R8
3.2	**CalculateDiscount**	Function	subtotal	discount (Real)	Returns 10% of the subtotal if it is over £20, otherwise £0.00	R9
3.3	**GetDeliveryCharge**	Function	delivery	deliveryCharge (Real)	Returns £2.50 if delivery is "Yes", otherwise £0.00	R10
3.4	**CalculateFinalTotal**	Function	subtotal, discount, deliveryCharge	finalTotal (Real)	Calculates: (subtotal - discount) + deliveryCharge	R11
4. Itemised bill						
4.1	**DisplayCustomerHeader**	Procedure	customerDetails[]	Displayed text	Prints the customer's details at the top of the bill	R12
4.2	**DisplayItemisedPizzas**	Procedure	numPizzas, sizes[], toppings[], sizePrices[], toppingCharges[], pizzaCosts[]	Displayed text	Prints one line per pizza showing its size, standard cost, extra topping cost, and total cost	R12
4.3	**DisplayTotals**	Procedure	subtotal, discount, deliveryCharge	Displayed text	Prints the subtotal, discount, and delivery charge	R12
4.4	**DisplayFinalTotal**	Procedure	finalTotal	Displayed text	Prints the final total to 2 decimal places	R11

3.2.4. Design Decisions

The subroutines were designed around the following decisions:

- **One job per subroutine**. Each subroutine does a single task, such as getting the pizza count or working out the discount, which keeps them short and means that each one can be tested on its own
- **Reuse**. GetSizePrice and GetToppingCharge are called inside a loop, so the pricing rules are written only once.
- **Validation at input**. The 1 to 6 check sits inside GetPizzaCount, so invalid data never reaches the calculations
- **Order of calculation**. the discount is subtracted from the subtotal before the delivery charge is added so delivery is never discounted.
- **Traceability**. The reference numbers (1.1 to 4.4) and requirement IDs match the structure chart which links each requirement to the design and testing.

P.T.O

3.3. Main Flowchart Design

The Main flowchart calls the subroutines in order. There are 15 in total as outlined in 3.2. Main calls 13 of them directly, and CalculatePizzaCosts calls the other two, GetSizePrice and GetToppingCharge, once for each pizza.

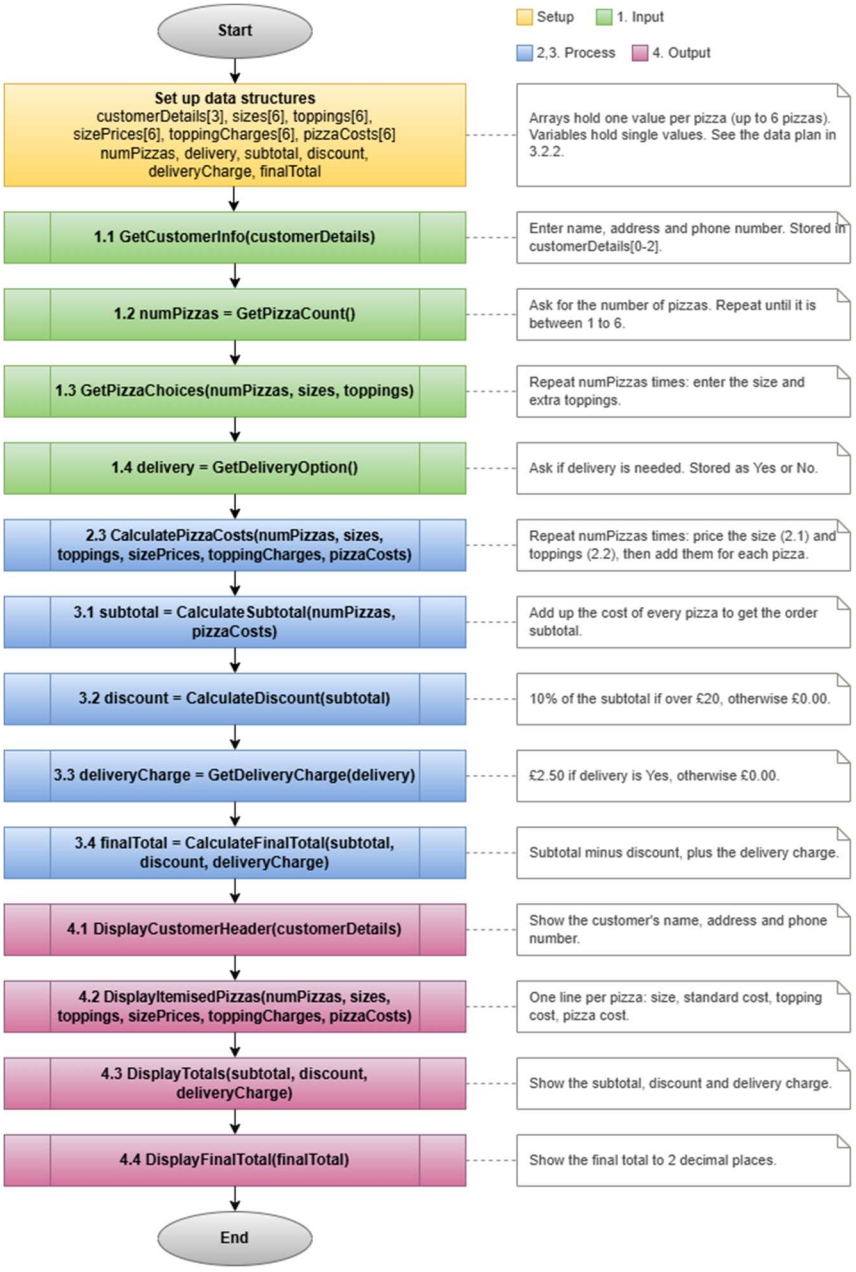


Figure 2: Main flowchart design (draw.io)

The dashed boxes on the right of the flowchart are notes explaining each step. This is abstraction: Main doesn't need to know how a subroutine works, only what it does.

3.4. Pseudocode (Generated with Flowgorithm)

The pseudocode below sets out the steps of every subroutine in structured English. It follows the PennineFlowgorithm_V4 file before the fixes in 5.2.

Main

```
Function Main
    Declare String Array customerDetails[3]
    Declare String Array sizes[6]
    Declare Integer Array toppings[6]
    Declare Real Array sizePrices[6], toppingCharges[6], pizzaCosts[6]
    Declare Integer numPizzas
    Declare String delivery
    Declare Real subtotal, discount, deliveryCharge, finalTotal

    Call GetCustomerInfo(customerDetails)
    Assign numPizzas = GetPizzaCount()
    Call GetPizzaChoices(numPizzas, sizes, toppings)
    Assign delivery = GetDeliveryOption()
    Call CalculatePizzaCosts(numPizzas, sizes, toppings, sizePrices, toppingCharges,
pizzaCosts)
    Assign subtotal = CalculateSubtotal(numPizzas, pizzaCosts)
    Assign discount = CalculateDiscount(subtotal)
    Assign deliveryCharge = GetDeliveryCharge(delivery)
    Assign finalTotal = CalculateFinalTotal(subtotal, discount, deliveryCharge)
    Call DisplayCustomerHeader(customerDetails)
    Call DisplayItemisedPizzas(numPizzas, sizes, toppings, sizePrices, toppingCharges,
pizzaCosts)
    Call DisplayTotals(subtotal, discount, deliveryCharge)
```

GetCustomerInfo

```
Function GetCustomerInfo (String Array customerDetails)
    Output "Enter customer name: "
    Input customerDetails[0]
    Output "Enter customer address: "
    Input customerDetails[1]
    Output "Enter customer phone number: "
    Input customerDetails[2]
End
```

GetPizzaCount

```
Function GetPizzaCount
    Declare Integer count

    Output "Enter number of pizzas (1-6): "
    Input count
    While count < 1 or count > 6
        Output "Invalid. Enter a number between 1 and 6: "
        Input count
    End
    Return Integer count
End
```

GetPizzaChoices

```
Function GetPizzaChoices (Integer numPizzas, String Array sizes, Integer Array toppings)
    Declare Integer counter

    For counter = 0 to numPizzas - 1
        Output "Enter size for pizza " & (counter + 1) & " (Small, Medium or Large): "
        Input sizes[counter]
        While sizes[counter] != "Small" and sizes[counter] != "Medium" and sizes[counter] !=
"Large"
            Output "Invalid size. Please type Small, Medium or Large: "
            Input sizes[counter]
        End
           Output "Enter number of extra toppings for pizza " & (counter + 1) & ": "
        Input toppings[counter]
        While toppings[counter] < 0
            Output "Toppings cannot be negative. ENTER AGAIN !!!: "
            Input toppings[counter]
        End
    End
End
```

GetDeliveryOption

```
Function GetDeliveryOption
    Declare String deliveryChoice

    Output "Is delivery required? (Yes or No): "
    Input deliveryChoice
    While deliveryChoice != "Yes" And deliveryChoice != "No"
        Output "Invalid. Please type Yes or No: "
        Input deliveryChoice
    End
    Return String deliveryChoice
End
```

GetSizePrice

```
Function GetSizePrice (String size)
    Declare Real sizePrice

    If size == "Small"
        Assign sizePrice = 3.25
    Else
        If size == "Medium"
            Assign sizePrice = 5.50
        Else
            Assign sizePrice = 7.15
        End
    End
```

GetToppingCharge

```
Function GetToppingCharge (Integer toppingCount)
    Declare Real toppingCharge

    If toppingCount == 0
        Assign toppingCharge = 0
    Else
        If toppingCount == 1
            Assign toppingCharge = 0.75
        Else
            If toppingCount == 2
                Assign toppingCharge = 1.35
            Else
                If toppingCount == 3
                    Assign toppingCharge = 2.00
                Else
                    Assign toppingCharge = 2.50
                End
            End
        End
    End
    Return Real toppingCharge
End
```

CalculatePizzaCosts

```
Function CalculatePizzaCosts (Integer numPizzas, String Array sizes, Integer Array toppings,
Real Array sizePrices, Real Array toppingCharges, Real Array pizzaCosts)
    Declare Integer counter

    For counter = 0 to numPizzas - 1
        Assign sizePrices[counter] = GetSizePrice(sizes[counter])
        Assign toppingCharges[counter] = GetToppingCharge(toppings[counter])
        Assign pizzaCosts[counter] = sizePrices[counter] + toppingCharges[counter]
    End
End
```

CalculateSubtotal

```
Function CalculateSubtotal (Integer numPizzas, Real Array pizzaCosts)
    Declare Real subtotal
    Declare Integer counter

    Assign subtotal = 0
    For counter = 0 to numPizzas - 1
        Assign subtotal = subtotal + pizzaCosts[counter]
    End
    Return Real subtotal
End
```

CalculateDiscount

```
Function CalculateDiscount (Real subtotal)
    Declare Real discount

    If subtotal > 20
        Assign discount = subtotal * 0.10
    Else
        Assign discount = 0
    End
    Return Real discount
End
```

GetDeliveryCharge

```
Function GetDeliveryCharge (String delivery)
    Declare Real deliveryCharge

    If delivery == "Yes"
        Assign deliveryCharge = 2.50
    Else
        Assign deliveryCharge = 0
    End
    Return Real deliveryCharge
End
```

CalculateFinalTotal

```
Function CalculateFinalTotal (Real subtotal, Real discount, Real deliveryCharge)
    Declare Real finalTotal

    Assign finalTotal = (subtotal - discount) + deliveryCharge
    Return Real finalTotal
End
```

DisplayCustomerHeader

```
Function DisplayCustomerHeader (String Array customerDetails)
    Output "PENNINE PIZZA WORKS - ORDER BILL"
    Output "Customer: " & customerDetails[0]
    Output "Address: " & customerDetails[1]
    Output "Phone: " & customerDetails[2]
End
```

DisplayItemisedPizzas

```
Function DisplayItemisedPizzas (Integer numPizzas, String Array sizes, Integer Array toppings,
Real Array sizePrices, Real Array toppingCharges, Real Array pizzaCosts)
    Declare Integer counter

    For counter = 0 to numPizzas - 1
        Output "Pizza " & (counter + 1) & ": " & sizes[counter] & ", " & toppings[counter] & "
extra toppings"
        Output " Standard £" & sizePrices[counter] & " Toppings £" & toppingCharges[counter] &
" Pizza cost £" & pizzaCosts[counter]
    End
End
```

DisplayTotals

```
Function DisplayTotals (Real subtotal, Real discount, Real deliveryCharge)
    Output "Subtotal: £" & ToFixed(subtotal, 2)
    Output "Discount: -£" & ToFixed(discount, 2)
    Output "Delivery charge: £" & ToFixed(deliveryCharge, 2)
End
```

DisplayFinalTotal

```
Function DisplayFinalTotal (Real finalTotal)
    Output "FINAL TOTAL: £" &  ToFixed(finalTotal, 2)
End
```

4. Implementation

This is the build stage of the project. We built the design from Section 3in Flowgorithm in stages, with each member adding their subroutines until all 15 were completed, using versions. The final program has 16 functions: Main plus the 15 subroutines from the subroutine list in 3.2.

4.1. Flowgorithm Version History

The Flowgorithm program was built in stages. Each team member downloaded the latest version from GitHub, added their own subroutines, saved it as the next version number and uploaded it again. This kept everyone's work visible, stopped anyone overwriting someone else's work, and meant the version number showed how far the program had got.

Version	Built By	What was Added
V1	Amaka	Main and the input subroutines 1.1 to 1.4 (**GetCustomerInfo**, **GetPizzaCount**, **GetPizzaChoices**, **GetDeliveryOption**)
V2	Enrique	Pizza cost subroutines 2.1 to 2.3 (**GetSizePrice**, **GetToppingCharge**, **CalculatePizzaCosts**)
V3	Toby	Order total subroutines 3.1 to 3.4 (**CalculateSubtotal**, **CalculateDiscount**, **GetDeliveryCharge**, **CalculateFinalTotal**)
V4	Fatimah	Itemised bill subroutines 4.1 to 4.4 (**DisplayCustomerHeader**, **DisplayItemisedPizzas**, **DisplayTotals**, **DisplayFinalTotal**)
Final Version	Amaka	Fixes from testing (see 5.2) and other improvements

4.2. Program Flowcharts

Each subsection below shows one function from the Flowgorithm file. Colours in Flowgorithm: yellow = declare/assign, blue = input, green = output, purple = call, diamond = decision, hexagon = loop.

4.2.1. Main

Declares the six arrays and the variables, then calls the subroutines in order, as in the draw.io design in 3.3.

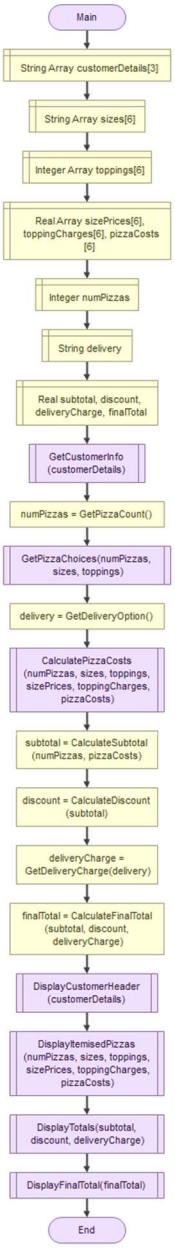


Figure 3: Main flowchart in Flowgorithm

4.2.2. GetCustomerInfo

Asks for name, address and phone and stores them in customerDetails[0], [1] and [2].

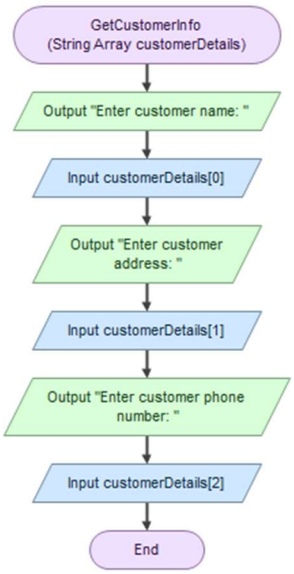


Figure 4: GetCustomerInfo flowchart in Flowgorithm

4.2.3. GetPizzaCount

Asks for the number of pizzas and loops until the value is between 1 and 6. Returns count.

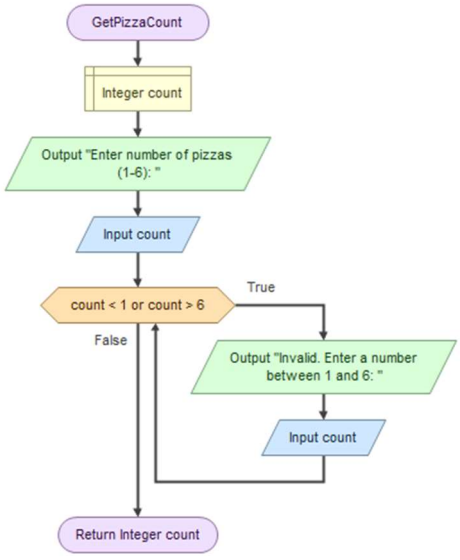


Figure 5: GetPizzaCount flowchart in Flowgorithm

4.2.4. GetPizzaChoices

A For loop that runs once per pizza. Each time it validates the size (Small, Medium, Large) and checks toppings are not negative.

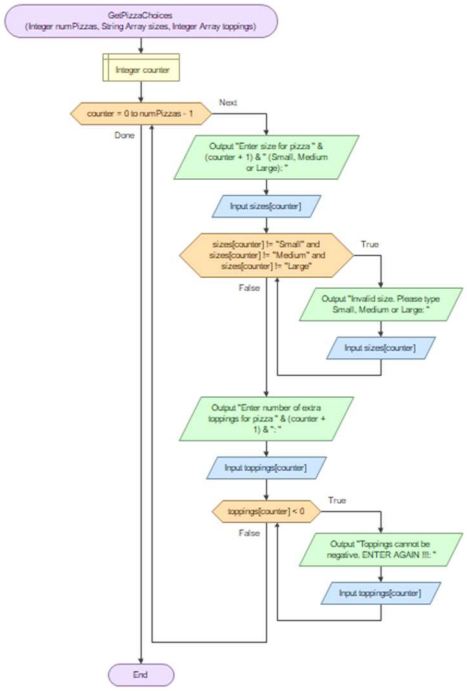


Figure 6: GetPizzaChoices flowchart in Flowgorithm

4.2.5. GetDeliveryOption

Asks "Yes or No" and loops until the answer is one of them. Returns deliveryChoice.

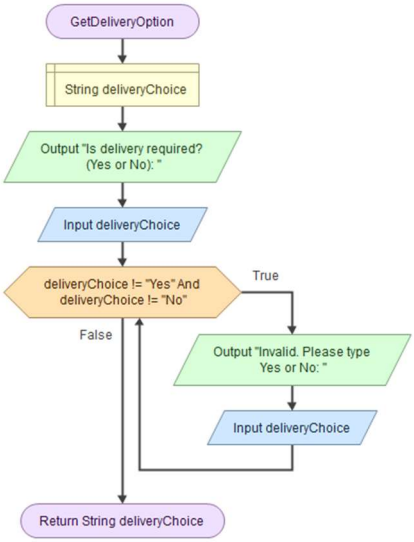


Figure 7: GetDeliveryOption flowchart in Flowgorithm

4.2.6. GetSizePrice

Nested If statements to match the size to £3.25, £5.50 or £7.15. Returns sizePrice.

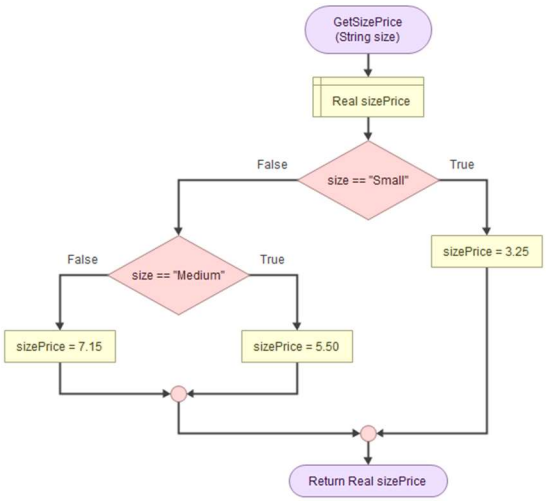


Figure 8: GetSizePrice flowchart in Flowgorithm

4.2.7. GetToppingCharge

Nested If statements: 0 = £0.00, 1 = £0.75, 2 = £1.35, 3 = £2.00, anything else = £2.50. Returns toppingCharge.

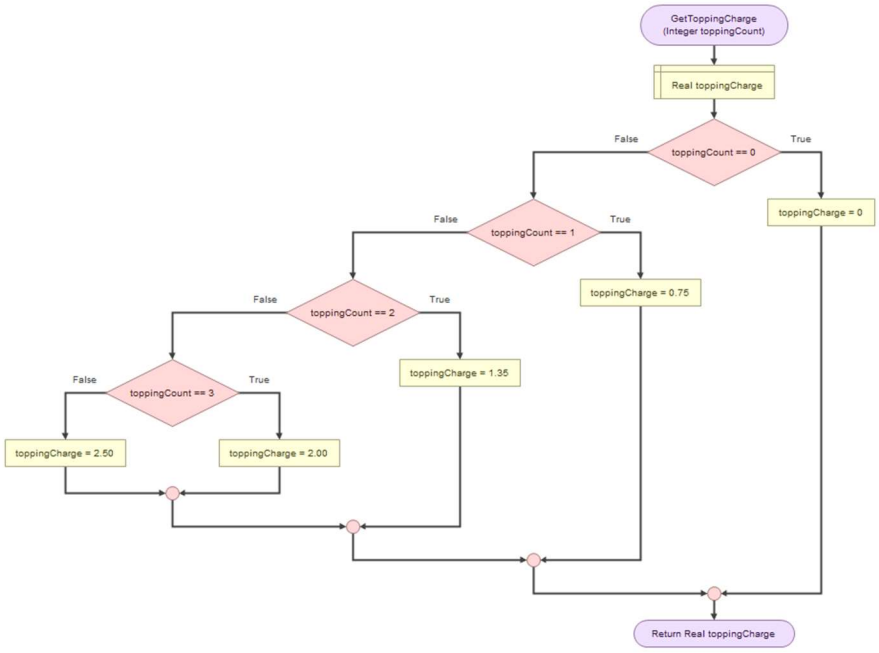


Figure 9: GetToppingCharge flowchart in Flowgorithm

4.2.8. CalculatePizzaCosts

Loops through each pizza, calls GetSizePrice and GetToppingCharge, stores both and adds them to give pizzaCosts[counter].

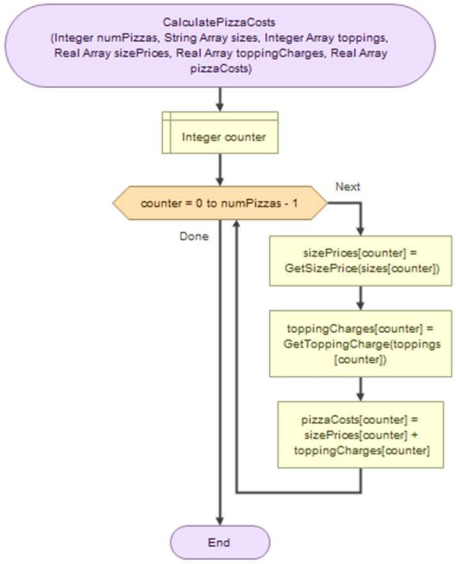


Figure 10: CalculatePizzaCosts flowchart in Flowgorithm

4.2.9. CalculateSubtotal

Starts at 0 and adds each pizzaCosts[counter]. Returns subtotal.

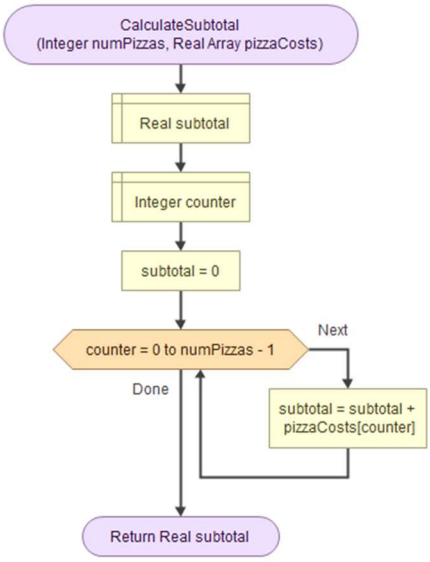


Figure 11: CalculateSubtotal flowchart in Flowgorithm

4.2.10. CalculateDiscount

If the subtotal is over 20, the discount is 10% of it, otherwise 0. Returns discount.

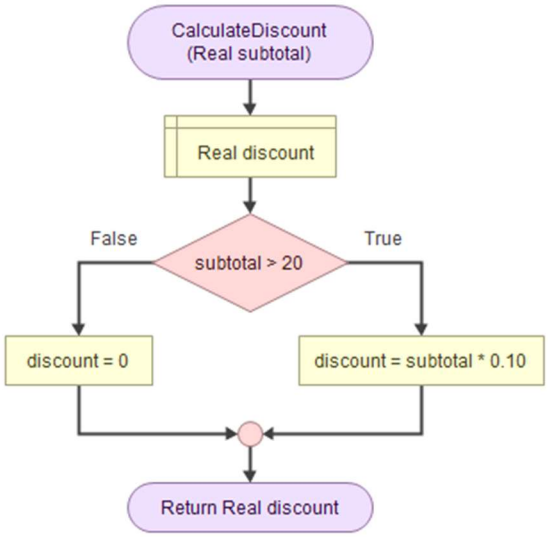


Figure 12: CalculateDiscount flowchart in Flowgorithm

4.2.11. GetDeliveryCharge

Returns 2.50 if delivery is "Yes", otherwise 0.

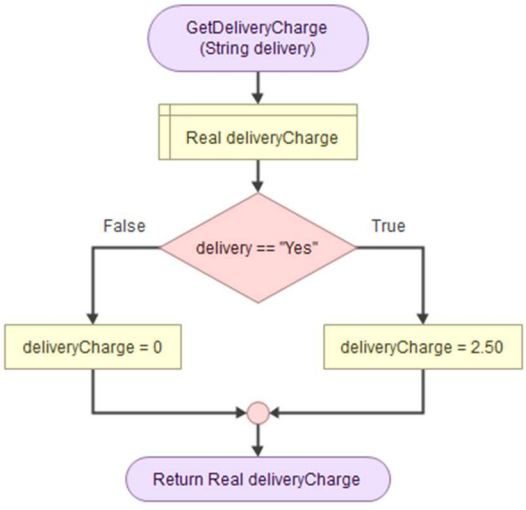


Figure 13: GetDeliveryCharge flowchart in Flowgorithm

4.2.12. CalculateFinalTotal

Calculates (subtotal - discount) + deliveryCharge. Returns finalTotal.

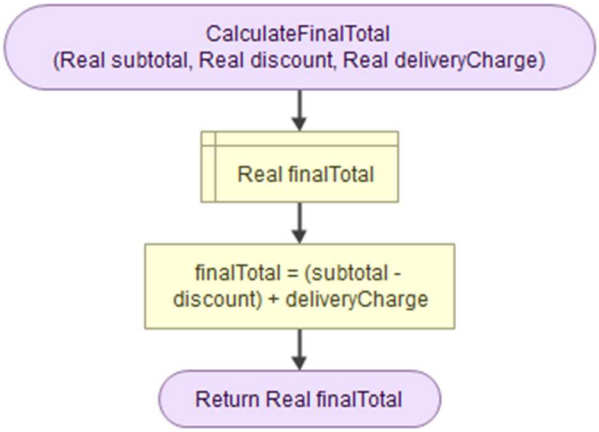


Figure 14: CalculateFinalTotal flowchart in Flowgorithm

4.2.13. DisplayCustomerHeader

Prints the bill title and the customer's name, address and phone.

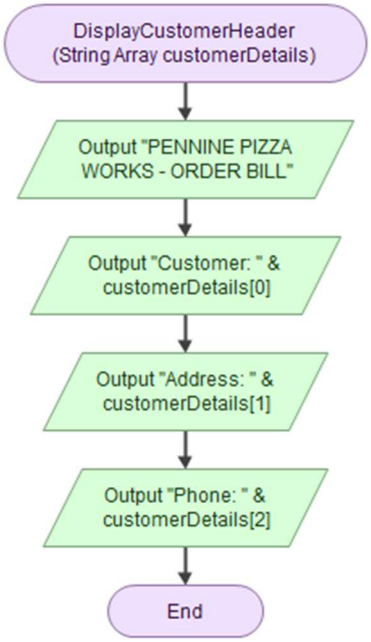


Figure 15: DisplayCustomerHeader flowchart in Flowgorithm

4.2.14. DisplayItemisedPizzas

Loops through the pizzas and prints two lines for each: the size and toppings, then the standard price, topping cost and pizza cost.

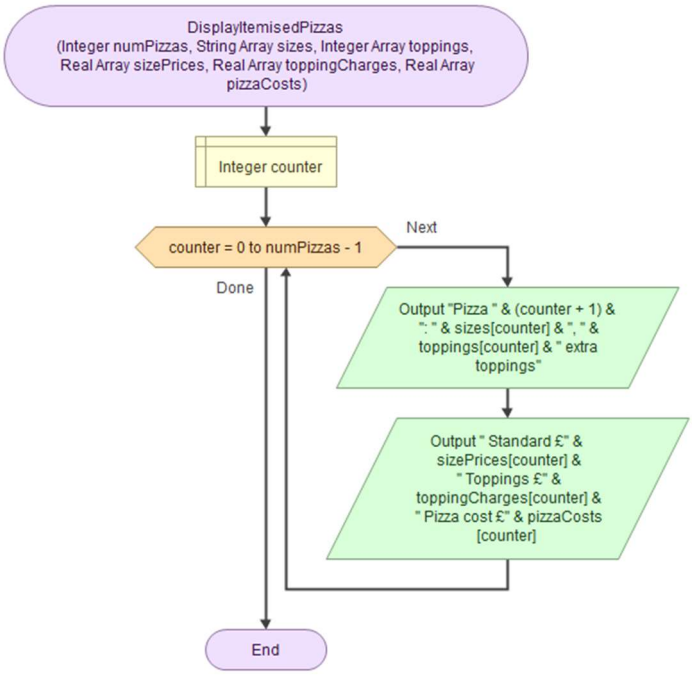


Figure 16: DisplayItemisedPizzas flowchart in Flowgorithm

4.2.15. DisplayTotals

Prints the subtotal, discount and delivery charge using ToFixed(value, 2).

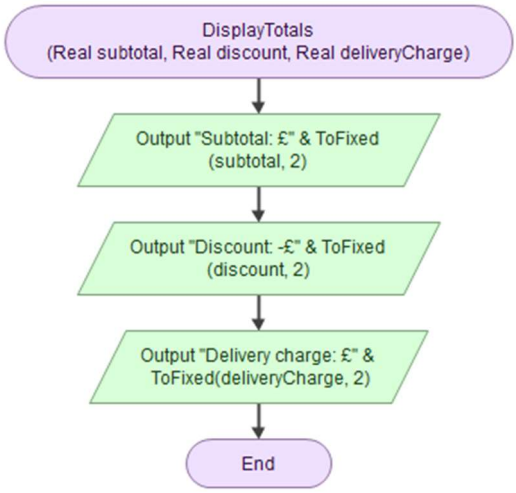


Figure 17: DisplayTotals flowchart in Flowgorithm

4.2.16. DisplayFinalTotal

Prints the final total to 2 decimal places.

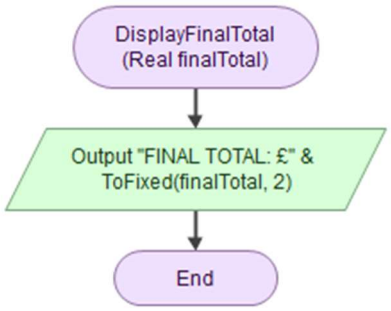


Figure 18: DisplayFinalTotal flowchart in Flowgorithm

5. Testing

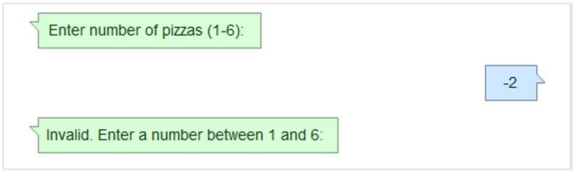
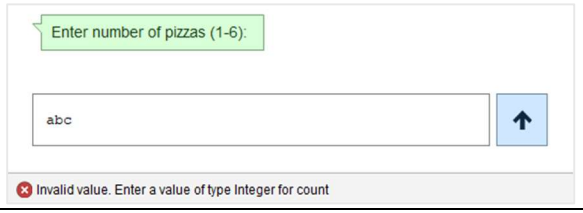
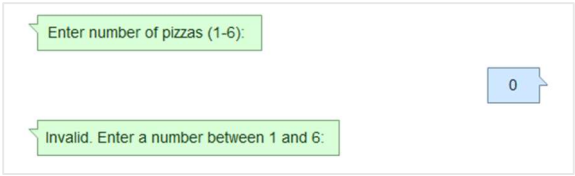
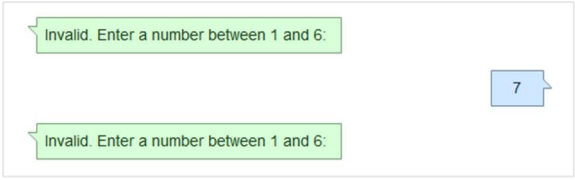
Three kinds of data were tested:

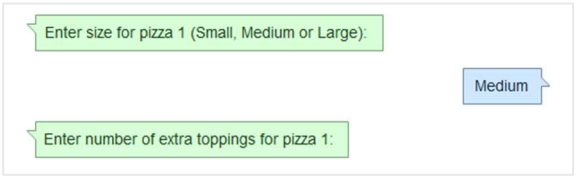
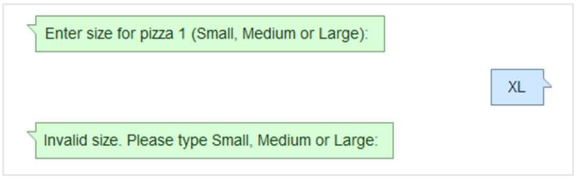
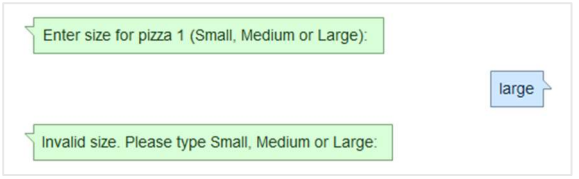
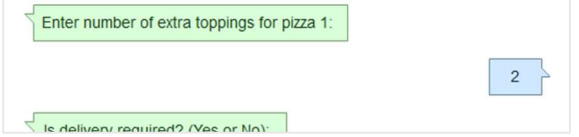
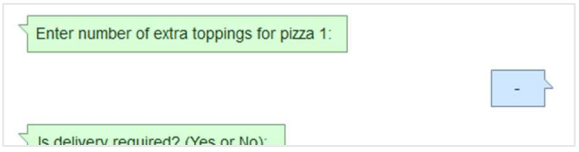
- **Normal**: typical orders the program should accept.
- **Extreme (boundary)**: values at and just beyond the limits, such as 1 and 6 pizzas, or a subtotal of exactly £20.00.
- **Abnormal**: input that should be rejected, such as the wrong size, wrong delivery word, letters instead of numbers, or negative numbers.

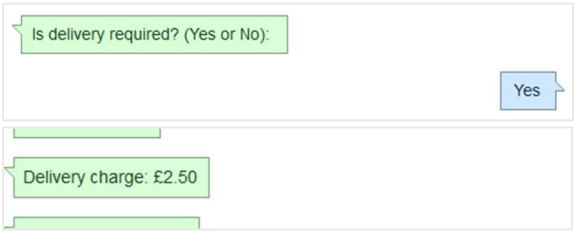
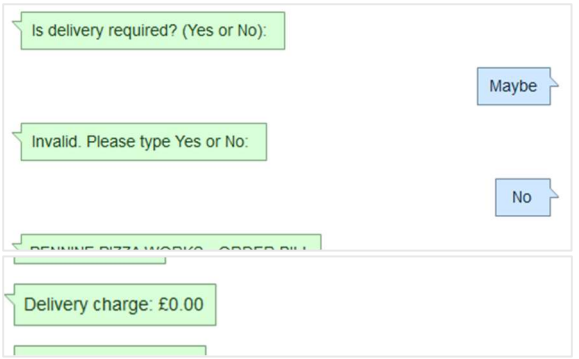
The expected results were worked out by hand before running each test. The tests were run on V4, the two failures were fixed in the final version (see 5.2) and re-tested.

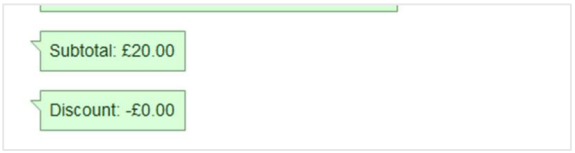
P.T.O

5.1. Test Table

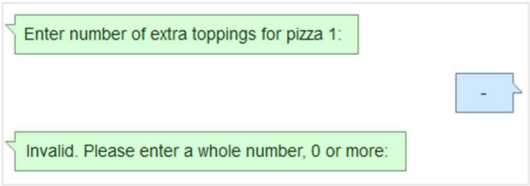
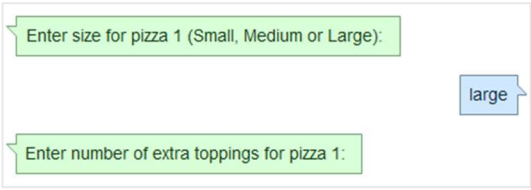
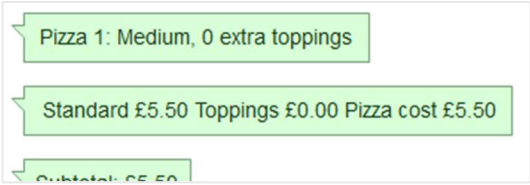
Test Case	Type	Purpose	Test Data	Expected Result	Actual Result	Status
Number of pizzas						
TC001	Normal	To test that a normal valid number is accepted	**3**	Should accept, then ask for pizza 1's size		**PASS**
TC002	Abnormal	To test that a negative number is rejected	**-2**	Should show an error message and ask again		**PASS**
TC003	Abnormal	To test that letters don't crash the program	**abc**	Should reject input as data type must be an integer		**PASS**
TC004	Extreme	To test that a number just below the limit is rejected	**0**	Should show an error message and ask again		**PASS**
TC005	Extreme	To test that a number just above the limit is rejected	**7**	Should show an error message and ask again		**PASS**

Pizza size						
TC006	Normal	To test that a valid pizza size is accepted	**Medium**	Should accept, then ask for the number of toppings		**PASS**
TC007	Abnormal	To test that an invalid pizza size is rejected	**XL**	Should show an error message and ask again		**PASS**
TC008	Abnormal	To test that a size typed in lower case is not rejected unfairly	**large**	Should accept, as staff shouldn't have to use capital letters		**FAIL**
Extra toppings						
TC009	Normal	To test that a valid number of toppings is accepted	**2**	Should accept, then move on		**PASS**
TC010	Abnormal	To test that a negative number of toppings is rejected	**-1**	Should show an error message and ask again		**PASS**
TC011	Abnormal	To test that a non-number is rejected for toppings (a bug we already found)	**-**	Should show an error message and ask again		**FAIL**

Delivery						
TC012	Normal	To test that choosing delivery adds the £2.50 charge	**Yes**	Delivery charge should be £2.50		**PASS**
TC013	Abnormal & Normal	To test that an invalid delivery answer is rejected, and that choosing no delivery gives a charge of £0.00	**Maybe**, then **No**	Should show an error message, then accept No. Delivery charge should be £0.00		**PASS**
Pizza Prices						
TC014	Normal	To test that a small pizza with no toppings is priced correctly	pizza: **1**, size: **Small**, toppings: **0**, delivery: **No**	Standard £3.25, Toppings £0, Pizza cost £3.25		**PASS**
TC015	Normal	To test that a medium pizza with toppings is priced correctly	pizza: **1**, size: **Medium**, toppings: **2**, delivery: **No**	Standard £5.5, Toppings £1.35, Pizza cost £6.85		**PASS**
TC016	Normal & Extreme	To test that a large pizza is priced correctly and 4 or more toppings are capped at £2.50	pizza: **1**, size: **Large**, toppings: **10**, delivery: **No**	Standard £7.15, Toppings £2.5, Pizza cost £9.65		**PASS**

Discount						
TC017	Extreme	To test that a subtotal of exactly £20.00 gets no discount	pizza: **4**, size/toppings: **Small/0**, **Small/0**, **Medium/0**, **Medium/4**, delivery: **No**	Subtotal £20.00, Discount -£0.00		**PASS**
TC018	Extreme	To test that a subtotal just over £20.00 gets 10% off	pizza: **3**, size/toppings: **Small/0**, **Large/1**, **Large/3**, delivery: **No**	Subtotal £20.30, Discount -£2.03		**PASS**
Whole system						
TC019	Normal	To test that a typical order works from start to finish with the discount and delivery working in the right order	pizza: **3**, size/toppings **Large/3**, **Large/2**, **Medium/1**, delivery: **Yes**	Itemised bill showing customer details and order details including: Subtotal £23.90, Discount -£2.39, FINAL TOTAL £24.01		**PASS**

5.2. Issues Found and Fixes

Found by	Issue	Fix	Status	Evidence
TC011	"-" was accepted as the number of toppings	Change **GetPizzaChoices** so toppings are read in as text instead of straight into a number. Add a loop to check every character is a digit from 0 to 9 using Len, Char and ToCode, and also reject an empty entry. Once the entry is valid, convert it with with ToInteger and store it in toppings[counter]	Fixed	
TC008	A size typed in lower case ("large") is rejected	Change **GetPizzaChoices** so the size is read in as text (sizeInput) instead of straight into sizes[counter]. Add a loop with three nested If statements to accept the full word in capitals or lower case, or just the first letter (S, M or L, in either case).	Fixed	
TC014 to TC016	The price lines on the bill show £5.5 and £0 instead of two decimal places.	Change **DisplayItemisedPizzas** by wrapping sizePrices[counter], toppingCharges[counter] and pizzaCosts[counter] in ToFixed(..., 2), so prices show as £5.50 and £0.00 instead of £5.5 and £0	Fixed	

5.3. Limitations and Future Improvements

- Prices are fixed in the code. If the shop changes its prices, someone has to edit **GetSizePrice**, **GetToppingCharge**, the discount and the delivery charge. A better version would keep prices in a file or database that staff can update.
- Customer details aren't validated. The name, address and phone number can be left blank. No check on the phone number format. Any text is accepted.
- Only one order per run. The program stops after printing the bill and doesn't save orders. A future version could save each bill to a file or keep an order history.